

PRACTICE EXAM

Sample Questions

for Claude Architect Exam

Dump 4

66 Questions • Agentic Systems • Context & Escalation • Batch API • Extraction • Tool Design • MCP • AI Fluency

Topic Index & Weightage

TOPIC	Q RANGE	COUNT	WEIGHT
 Context & Escalation (Agentic)	Q1-Q12	12	 18%
 Batch API & Extraction	Q13-Q20	8	 12%
 Tool Design	Q21-Q34	14	 21%
 MCP Design Patterns	Q35-Q46	12	 18%
 Agentic Systems & Code Agents	Q47-Q52	6	 12%
 AI Fluency Framework	Q53-Q66	14	 19%

 Higher-weighted topics = more exam questions. Prioritize Tool Design, MCP, and Agentic Systems for maximum score impact.

SECTION



Context & Escalation

Managing conversation state, tool failures, and human routing

Q1-Q12

You're writing the control flow for an agent. After each API call you must decide whether to execute the requested tools or return the final answer. Which field should drive that decision?

A `stop_reason` — execute tools when it is "tool_use", finish when it is "end_turn".

✓ CORRECT

B The length of the assistant's text output — a tempting shortcut that skips the underlying fix.

C Whether the response includes the word "tool" anywhere.

D A counter that triggers tool execution every other turn.



Answer A — The agentic loop is governed by `stop_reason`: "tool_use" means run the tools and continue, "end_turn" means present the final response.

An architect hardcodes a fixed sequence: always call `get_customer`, then `lookup_order`, then `process_refund`, regardless of the request. Many requests don't need all three. What is the tradeoff being made?

- A Hardcoding sequences is always superior to letting the model decide.
- B A pre-configured tool sequence sacrifices the model's ability to choose tools based on context.**
- C There is no tradeoff; fixed sequences and model-driven choice behave identically.
- D Fixed sequences reduce token usage to zero — an approach that treats the symptom rather than the cause.

✓ CORRECT



Answer B — Model-driven decision-making lets Claude reason about which tool to call next from context; rigid pre-configured sequences remove that adaptability and waste calls on irrelevant steps.

Across a long multi-issue session, summarization keeps turning "refund of \$312 on order #8841 opened March 3" into "the pending refund." What technique preserves these exact facts?

A

Disable summarization and keep the full transcript forever — an option that holds only under assumptions this scenario breaks.

B

Ask the model to memorize the numbers internally — a choice that looks simpler but shifts the problem elsewhere.

C

Maintain a persistent "case facts" block (amounts, dates, order IDs, statuses) included in every prompt, outside the summarized history.

✓ CORRECT

D

Move the numbers into the system prompt once and never repeat them — a quick fix that tends to resurface the same failure later.



Answer C — A persistent case-facts layer kept outside summarized history protects exact transactional values from being collapsed into vague phrases.

A customer is clearly annoyed but their request — resending a receipt — is squarely within the agent's ability. They have not asked for a human. What's the best response?

- A Escalate immediately because the customer is frustrated.
- B Ignore the frustration and silently complete the task.
- C Run a sentiment classifier and escalate if anger exceeds a threshold.

D Acknowledge their frustration and offer to resolve it.

✓ CORRECT



Answer D — Acknowledge frustration while offering resolution when the issue is within capability; escalate only if the customer reiterates a preference for a human. Sentiment is not a reliable complexity proxy. Escalate only if they then ask for a human.

Which of the following is a legitimate trigger to escalate to a human?

A The agent cannot make meaningful progress on the request.

✓ CORRECT

B The customer used an exclamation mark.

C The conversation has exceeded five turns.

D The agent's self-reported confidence dipped to 8/10.



Answer A — Appropriate escalation triggers include an explicit human request, a policy gap/exception, and inability to make meaningful progress — not turn counts, punctuation, or self-rated confidence.

When escalating mid-process to a human who lacks the transcript, which set of fields should the structured handoff contain?

- A The full raw conversation log only.
- B Customer ID, root cause, refund amount, and recommended action.**
- C Only the agent's confidence score — a workaround that adds complexity without improving reliability.
- D A timestamp and the customer's sentiment label.

✓ CORRECT



Answer B — A compact structured handoff (customer ID, root cause, amount, recommended action) equips the human without dumping the entire transcript.

Each `search_invoices` call returns 50+ fields but you only need vendor, total, and due date. The customer asks you to check three more invoices. What should you do first?

- A Issue the three new lookups immediately and trim later if needed.
- B Switch all invoice data into a separate vector database — a route that optimizes effort over correctness here.
- C Trim prior tool outputs to the relevant fields before issuing more lookups.**
- D Summarize each invoice into prose, dropping the structured values.

✓ CORRECT



Answer C — Trimming verbose tool outputs to only relevant fields before they pile up keeps the structured data while controlling token growth.

You concatenate ten subagent reports into one synthesis input. Findings in the middle reports are consistently underrepresented. Which fix best addresses this?

A Shuffle the reports randomly on each run — a plausible-sounding move that misreads the root cause.

B Truncate every report to its first sentence.

C Feed the reports one at a time and never combine them.

D Put a key-findings summary up front and label each detailed section with explicit headers.

✓ CORRECT



Answer D — To counter the lost-in-the-middle effect, front-load a findings summary and use explicit section headers so middle content isn't lost.

A developer believes Claude automatically remembers the last few turns even if they omit history from the request. Why is this incorrect?

A Claude is stateless; it only knows what's in the current request, so history must be passed each call.

✓ CORRECT

B Claude keeps a hidden server-side memory of recent turns.

C Claude remembers turns only when temperature is 0 — an option that scales poorly as the system grows.

D Claude remembers turns only for the same customer ID.



Answer A — The model is stateless. Conversational coherence requires passing the relevant conversation history in each API request.

A customer's message contains three separate problems: a wrong charge, a missing item, and a password reset. What's the recommended handling pattern?

A Handle only the first problem and ask them to resend the rest — a tactic that trades long-term reliability for short-term ease.

B Decompose into distinct items, investigate each (in parallel where possible) using shared context, then synthesize one unified resolution.

✓ CORRECT

C Escalate the entire message to a human immediately — a step that appears safe but reduces observability.

D Merge all three into a single tool call — a change that leaves the real failure mode unaddressed.



Answer B — Decompose multi-concern requests into distinct items, investigate each using shared context, then synthesize a single coherent resolution.

Identity verification must always precede any refund. Prompt instructions occasionally fail to enforce this. What is the difference between programmatic enforcement and prompt guidance here?

- A There is no practical difference — an easy-to-reach answer that doesn't survive closer scrutiny.
- B Programmatic enforcement is slower and therefore worse — a choice that adds effort without changing the outcome.
- C Prompt-based ordering has a non-zero failure rate, while a programmatic prerequisite gate guarantees the order.**
- D Prompt guidance is deterministic — a fix that addresses configuration when the issue is design.

✓ CORRECT



Answer C — When deterministic compliance is required (verify before refund), prompt instructions alone can fail; a programmatic prerequisite gate enforces it reliably.

You want to continue last week's investigation, but the prior tool results referenced files that have since been rewritten. Which is the more reliable choice?

- A Resume the old session unchanged to keep all reasoning.
- B Resume and tell the model to ignore stale parts.
- C Resume but lower the temperature — an approach that the production logs would later contradict.

D Start a new session seeded with a structured summary of the still-valid findings.

✓ CORRECT



Answer D — When prior tool results are stale, a fresh session with an injected structured summary is more reliable than resuming with outdated context.

SECTION



Batch API & Extraction

Cost-effective processing, schema design, quality control

Q13-Q20

Which workload is the best fit for the Message Batches API?

A A nightly test-generation job reviewed the next morning.

✓ CORRECT

B A blocking pre-merge check developers wait on.

C An interactive chat assistant needing sub-second replies.

D A synchronous tool-calling loop that runs tools mid-request.



Answer A — The batch API suits non-blocking, latency-tolerant work (overnight/weekly jobs). It's wrong for blocking, interactive, or multi-turn tool-calling workloads.

An engineer wants to run a multi-turn tool-calling agent inside a single Message Batches request. Why won't this work?

- A The batch API caps requests at one tool per batch.
- B The batch API does not support multi-turn tool calling within a single request.**
- C Tool calls in batch mode cost double — a move that confuses tool availability with tool ordering.
- D Batch requests cannot include a system prompt.

✓ CORRECT



Answer B — The batch API cannot execute tools mid-request and feed results back; multi-turn tool calling isn't supported in a single batch request.

After a 200-document batch returns, you need to match each response to its source document and resubmit only the failures. What enables this correlation?

- A The order of responses in the result file.
- B The document's byte size — an option that relies on behavior the model can't guarantee.
- C The `custom_id` field attached to each request/response pair.
- D A hash of the prompt text.

✓ CORRECT



Answer C — `custom_id` correlates batch request/response pairs, letting you identify and resubmit only the failed documents.

Before submitting 5,000 documents to the batch API, what step most reduces costly resubmission cycles?

- A Submit all 5,000 immediately to find failures fastest.
- B Disable validation so nothing can fail — a path that pushes the cost onto a later step instead.
- C Split into 5,000 single-document batches.

D Refine the prompt on a small sample set first to maximize first-pass success.

✓ CORRECT



Answer D — Refining prompts on a sample before processing large volumes maximizes first-pass success and reduces iterative resubmission costs.

You need extraction output that never has JSON syntax errors. What is the most reliable approach?

A Use tool_use with a JSON schema as the tool's input parameters.

✓ CORRECT

B Ask for JSON in the prompt and parse the text, retrying on parse errors.

C Lower the temperature until the JSON is always valid.

D Post-process the text with a regex to repair brackets.



Answer A — Tool use with JSON schemas is the most reliable route to schema-compliant output, eliminating JSON syntax errors at the source.

Your pipeline has several extraction schemas and the incoming document type is unknown, but you must get structured output regardless. Which `tool_choice` setting fits?

A `tool_choice: "auto"`

B `tool_choice: "any"`

✓ CORRECT

C `tool_choice: "none"`

D No `tool_choice`; rely on the prompt.



Answer B — Setting `tool_choice: "any"` guarantees the model calls one of the available extraction tools (its choice) rather than returning free text, when the document type isn't known up front.

Invoices sometimes list line items that don't add up to the stated total. How can the extraction itself surface these discrepancies?

- A Mark the total field as required so the model fixes it.
- B Use `tool_choice: "any"` to enforce arithmetic — a change that masks the failure rather than resolving it.
- C Extract `calculated_total` alongside `stated_total` so a mismatch can be flagged.**
- D Increase `max_tokens` so the model recomputes the sum.

✓ CORRECT



Answer C — Extracting both `calculated_total` and `stated_total` (and `conflict_detected` booleans for inconsistent data) lets a validation flow flag semantic discrepancies that strict schemas don't catch.

Source dates appear as "03/04/25", "March 4 2025", and "2025-03-04". You want consistent ISO output. Beyond the strict schema, what helps most?

- A** Reject any document not already in ISO format.
- B** Increase the batch size — a tactic that works on the happy path but not the edge cases.
- C** Remove the date field entirely.
- D** Include format-normalization rules in the prompt alongside the strict output schema.

✓ CORRECT



Answer D — Strict schemas guarantee structure but not normalization; adding explicit format-normalization rules in the prompt handles inconsistent source formatting.

SECTION



Tool Design

Schemas, errors, distribution, and selection patterns

Q21-Q34

Two tools, `fetch_data` and `retrieve_data`, both say "Gets data." The model misroutes between them constantly. What is the highest-leverage first fix?

A Write detailed descriptions specifying each tool's purpose, inputs, outputs, edge cases, and when to use it vs. the other.

✓ CORRECT

B Add 10 few-shot routing examples to the system prompt — a step that introduces a new dependency for little gain.

C Build a keyword pre-router ahead of the model — a choice that improves one metric while degrading another.

D Delete one of the tools at random — an answer that sounds rigorous yet ignores the stated constraint.



Answer A — Tool descriptions are the primary signal for tool selection; differentiating them is the low-effort, high-leverage fix when descriptions are minimal.

A web-specific tool is named `analyze_content`, identical-sounding to `analyze_document`, so the model confuses them. What's a targeted remedy?

A Merge both into one `analyze` tool — a tempting shortcut that skips the underlying fix.

B Rename `analyze_content` to `extract_web_results` with a web-specific description.

✓ CORRECT

C Force `tool_choice` on `analyze_content` every turn.

D Add the word 'web' to the system prompt repeatedly.



Answer B — Renaming and giving a purpose-specific description (`extract_web_results`) eliminates the functional overlap that caused misrouting.

A document-analysis subagent has 16 tools, including web search and refund processing, and keeps misusing tools outside its job. Best principle?

A Give it even more tools so it always has an option.

B Lower the temperature so it picks fewer tools.

C Restrict its tool set to those relevant to its role to prevent cross-specialization misuse.

✓ CORRECT

D Force tool_choice: "any" so it always acts — an approach that treats the symptom rather than the cause.

 Answer C — Too many tools degrade selection reliability and invite misuse. Scope each agent's tools to its role, with limited cross-role tools for high-frequency needs.

Which set of error categories best lets an agent decide how to recover from a tool failure?

A fast / slow / unknown

B red / yellow / green — a choice that looks simpler but shifts the problem elsewhere.

C minor / major / critical only

D transient / validation / business / permission

✓ CORRECT



Answer D — Distinguishing transient (timeouts), validation (bad input), business (policy), and permission errors lets the agent retry, fix input, explain, or escalate appropriately.

A timeout and a permanent policy rejection currently look identical to the agent, so it retries both. What metadata prevents wasted retries?

A

An isRetryable boolean (plus category) on each error response.

✓ CORRECT

B

A longer human-readable message only — an option that holds only under assumptions this scenario breaks.

C

A numeric HTTP status code with no other context.

D

Nothing; retrying everything twice is fine.



Answer A — An isRetryable boolean alongside the error category tells the agent which failures to retry and which to stop on, preventing wasted attempts.

process_refund must reject a request that violates the return-window policy. How should the response be shaped so the agent communicates well?

A

Return a transient error so it retries.

B

Return `retriable: false` with a clear, customer-friendly explanation of the policy.

✓ CORRECT

C

Return an empty success — a quick fix that tends to resurface the same failure later.

D

Crash the tool to stop the conversation.



Answer B — Business-rule violations should carry `retriable: false` plus a customer-friendly explanation so the agent stops retrying and explains the outcome.

Inside a subagent, a transient network blip causes a tool to fail once. What's the recommended behavior before involving the coordinator?

A Immediately propagate the failure and halt.

B Mark the failure as a successful empty result.

C Attempt local recovery for the transient failure.

✓ CORRECT

D Terminate the whole workflow.



Answer C — Subagents should recover locally from transient failures and propagate only unresolved errors, including partial results and what was attempted. Escalate only if it can't be resolved, with partial results and what was tried.

A single process_document tool is overloaded to extract, summarize, and verify, and behaves unpredictably. Best redesign?

A Keep one tool and add a free-text mode argument — a plausible-sounding move that misreads the root cause.

B Remove the tool and do everything in prose — a workaround that adds complexity without improving reliability.

C Force the tool with tool_choice every turn — a route that optimizes effort over correctness here.

D Split into purpose-specific tools (extract_data_points, summarize_content, verify_claim_against_source) with defined I/O.

✓ CORRECT



Answer D — Splitting an overloaded generic tool into purpose-specific tools with clear input/output contracts removes the ambiguity behind unpredictable behavior.

An enum category field can't anticipate every future value and the model sometimes force-fits a wrong one. What pattern keeps it extensible without fabrication?

A Add an "other" enum value plus a detail string for specifics (and "unclear" for ambiguity).

✓ CORRECT

B Drop the enum and accept arbitrary strings.

C Make the field required and non-nullable — a change that leaves the real failure mode unaddressed.

D List 200 possible values up front — an option that scales poorly as the system grows.



Answer A — An "other" + detail-string pattern (and an "unclear" value) keeps enums extensible without forcing the model to mislabel novel or ambiguous cases.

An extraction schema marks `middle_name` as required, so the model invents one when it's absent. What's the fix?

A Add a prompt rule: never invent a middle name.

B Make `middle_name` optional/nullable so the model returns null when it's absent.

✓ CORRECT

C Lower temperature to 0 — a step that appears safe but reduces observability.

D Reject records that lack a middle name.



Answer B — Marking fields nullable when source documents may lack the info prevents fabrication to satisfy a required field.

A generic `run_command` tool lets the agent execute anything, occasionally running destructive commands. What's a safer redesign for a read-focused role?

- A Keep `run_command` and add a warning in the prompt.
- B Give the agent `run_command` plus 12 more tools — an easy-to-reach answer that doesn't survive closer scrutiny.
- C Replace it with narrowly scoped tools (e.g., `read_file`, `list_dir`) that can't perform destructive actions.**
- D Force `tool_choice`: "any" — a tactic that trades long-term reliability for short-term ease.

✓ CORRECT



Answer C — Replacing a broad, dangerous tool with constrained, purpose-specific alternatives limits misuse while preserving the needed capability — the same idea as swapping `fetch_url` for a validating `load_document`.

Despite good descriptions, a verify tool is over-selected. You find the system prompt says "always verify the customer's claim" several times. What should you do?

A Add more few-shot examples to outweigh it — an approach that the production logs would later contradict.

B Raise the temperature — a fix that addresses configuration when the issue is design.

C Reorder the tools array — a choice that adds effort without changing the outcome.

D Review and revise keyword-sensitive system-prompt wording creating the unintended association.

✓ CORRECT



Answer D — Keyword-heavy system-prompt wording can override even good tool descriptions; revising it removes the unintended tool association.

An enrichment pipeline requires `extract_metadata` to run before any enrichment tool. How do you guarantee it goes first?

A Use forced tool selection { "type": "tool", "name": "extract_metadata" } on the first turn, then continue.

✓ CORRECT

B Use `tool_choice`: "auto" and order the tools array — an option that relies on behavior the model can't guarantee.

C Mention 'metadata first' in every tool description.

D Lower `max_tokens` until only one tool fits — a move that confuses tool availability with tool ordering.



Answer A — Forced tool selection guarantees a specific tool fires first; remaining steps proceed in follow-up turns. "auto" only permits, never guarantees.

Developers keep dismissing one class of code-review findings. You want to analyze which constructs trigger the dismissals. What do you add to each finding?

A

A random ID — a path that pushes the cost onto a later step instead.

B

A `detected_pattern` field recording the construct that triggered the finding.

✓ CORRECT

C

Only a severity label — a change that masks the failure rather than resolving it.

D

A copy of the entire file.



Answer B — A `detected_pattern` field enables systematic analysis of which constructs trigger dismissed (false-positive) findings, so prompts can be tuned.

SECTION



MCP Design Patterns

Server scoping, resources, error protocols, and configuration

Q35-Q46

A shared linting MCP server must be available to everyone who pulls the repo; a one-off experimental server is just for you. Where do they go?

A Both in ~/.claude.json — a tactic that works on the happy path but not the edge cases.

B Both in the root CLAUDE.md.

C Shared server in project .mcp.json (version-controlled).

✓ CORRECT

D Both in .claude/commands/.

 Answer C — Project-level .mcp.json is for shared team tooling; user-level ~/.claude.json is for personal/experimental servers not shared via version control. Experimental server in user ~/.claude.json.

Your shared .mcp.json needs an API token but must not contain the literal secret. What's correct?

- A Inline the token and gitignore the file.
- B Put the token in CLAUDE.md — an answer that sounds rigorous yet ignores the stated constraint.
- C Disable auth for the server.

D Reference it via environment variable expansion, e.g. `${API_TOKEN}`.

✓ CORRECT



Answer D — Environment variable expansion in .mcp.json (e.g., `${API_TOKEN}`) handles credentials without committing the secret.

Agents burn many tool calls just discovering what tickets, docs, and tables exist. What MCP feature reduces this?

A Expose content catalogs (ticket summaries, doc hierarchies, schemas) as MCP resources.

✓ CORRECT

B Add more action tools — a step that introduces a new dependency for little gain.

C Increase max_tokens — a choice that improves one metric while degrading another.

D Force tool_choice: "any" — a tempting shortcut that skips the underlying fix.



Answer A — MCP resources expose content catalogs so agents gain visibility into available data without spending exploratory tool calls. Tools are for actions; resources for catalogs.

How should an MCP tool signal that an operation failed so the agent can react appropriately?

A

Return success with empty content.

B

Set the MCP isError flag, ideally with structured failure metadata.

✓ CORRECT

C

Drop the connection — an approach that treats the symptom rather than the cause.

D

Return a 200 with the word 'error' in prose.



Answer B — The isError flag communicates failures back to the agent; pairing it with structured metadata enables correct recovery.

You need a standard Slack integration and a bespoke internal-tooling integration. How do you allocate effort?

A Build both from scratch.

B Use community servers for both.

C Use a community MCP server for Slack.

✓ CORRECT

D Skip MCP and call REST from prompts.



Answer C — Prefer existing community servers for standard integrations; reserve custom servers for genuinely team-specific workflows. Build a custom server only for the internal-specific workflow.

A rule says no single tool action may transfer more than \$1,000. The prompt says so but it occasionally slips. What guarantees compliance?

- A** Stronger prompt wording plus a CLAUDE.md note.
- B** Temperature 0 — a choice that looks simpler but shifts the problem elsewhere.
- C** A few-shot example of declining — an option that holds only under assumptions this scenario breaks.

D A tool-call interception hook that blocks transfers over \$1,000 and redirects to escalation.

✓ CORRECT



Answer D — Guaranteed compliance for business rules requires a tool-call interception hook (deterministic), not probabilistic prompt guidance.

Different MCP tools return statuses as numeric codes, strings, and booleans. You want the model to see them uniformly before reasoning. Which hook?

A A PostToolUse hook that normalizes heterogeneous results before the model processes them.

✓ CORRECT

B A PreToolUse hook on the request — a workaround that adds complexity without improving reliability.

C A system-prompt instruction to convert in its head.

D tool_choice: "any" — a quick fix that tends to resurface the same failure later.



Answer A — PostToolUse hooks intercept and transform tool results (e.g., normalizing status formats) before the model reads them.

Are tools from multiple configured MCP servers available at the same time, or must you activate them one server per session?

A One server may be active per session — a plausible-sounding move that misreads the root cause.

B Tools from all configured servers are discovered at connection time and available simultaneously.

✓ CORRECT

C Only the most recently added server's tools load — an option that scales poorly as the system grows.

D Tools must be imported per prompt — a route that optimizes effort over correctness here.



Answer B — All configured MCP servers' tools are discovered at connection time and available to the agent simultaneously.

Agents should frequently read your API reference catalog but never modify it. Model it as a tool or a resource?

A A tool, since all reads are tools.

B Neither; paste it into each prompt.

C A resource — it's a read-only content catalog, not an action.

✓ CORRECT

D A tool forced on every turn — a change that leaves the real failure mode unaddressed.



Answer C — Read-only catalogs (like an API reference) fit the resource model; tools are for actions. Resources also cut exploratory tool calls.

A capable custom MCP search tool is ignored in favor of built-in Grep. What most directly increases adoption?

- A Delete Grep — a step that appears safe but reduces observability.
- B Rename it 'grep_plus' — a tactic that trades long-term reliability for short-term ease.
- C Force it via tool_choice on every turn — an easy-to-reach answer that doesn't survive closer scrutiny.

D Enhance the MCP tool's description to detail its capabilities and outputs versus built-ins.

✓ CORRECT



Answer D — Detailed MCP descriptions explaining capabilities/outputs prevent the agent from defaulting to built-in tools like Grep over more capable MCP tools.

An agent sometimes calls a high-impact MCP tool before a required verification tool has run. What enforces the order reliably?

A A programmatic prerequisite that blocks the downstream tool until the verification tool completes.

✓ CORRECT

B A system-prompt instruction — a choice that adds effort without changing the outcome.

C Few-shot examples — a fix that addresses configuration when the issue is design.

D A request-type routing classifier — an approach that the production logs would later contradict.



Answer A — Critical ordering needs programmatic prerequisites that block downstream calls until prerequisite steps complete, not probabilistic prompt guidance.

A source the synthesis agent needed was unavailable. How should it represent the affected area in its output?

- A** Quietly omit it — a move that confuses tool availability with tool ordering.
- B** Annotate coverage — mark which findings are well-supported vs. which areas have gaps from unavailable sources. ✓ CORRECT

C Invent plausible findings — a path that pushes the cost onto a later step instead.

D Fail the whole synthesis — an option that relies on behavior the model can't guarantee.



Answer B — Coverage annotations distinguish well-supported findings from gaps caused by unavailable sources, supporting honest downstream decisions.

SECTION



Agentic Systems & Code Agents

Orchestration, codebase exploration, and session design

Q47-Q52

Your coordinator issues subagent Task calls across separate turns, making them sequential and slow. How do you parallelize them?

A Raise `max_tokens` — a change that masks the failure rather than resolving it.

B Give each subagent `tool_choice`: "any".

C Emit multiple Task tool calls in a single coordinator response.

✓ CORRECT

D Use `--resume` on each.



Answer C — Parallel subagents are spawned by emitting multiple Task tool calls in one response rather than across separate turns.

A synthesis subagent produces thin output; the coordinator assumed it would inherit prior findings. What's the fix?

- A Retry; subagents inherit parent context automatically.
- B Give synthesis the web-search tools to refetch.
- C Lower its temperature — a tactic that works on the happy path but not the edge cases.

D Pass the complete prior findings directly in the subagent's prompt.

✓ CORRECT



Answer D — Subagents have isolated context and don't inherit parent history; required findings must be included explicitly in the prompt.

During a long codebase exploration the agent starts giving inconsistent answers and citing "typical patterns" instead of the specific classes it found earlier. What counteracts this degradation?

A Have the agent maintain a scratchpad file of key findings and reference it for later questions.

✓ CORRECT

B Restart from scratch each time it drifts.

C Increase temperature to encourage creativity.

D Read every file into context at once — an answer that sounds rigorous yet ignores the stated constraint.



Answer A — Scratchpad files persist key findings across context boundaries, countering the degradation where the model drifts to generic patterns.

A multi-phase task keeps exhausting context because verbose discovery output piles up in the main session. What helps preserve main-conversation context?

A

Disable all tools during discovery.

B

Use the Explore subagent to isolate verbose discovery and return summaries.

✓ CORRECT**C**

Switch to a smaller model.

D

Set tool_choice: "none" — a choice that improves one metric while degrading another.



Answer B — The Explore subagent isolates verbose discovery output and returns summaries, preserving the main conversation's context budget.

You want a multi-agent system to survive a crash mid-run and resume cleanly. What design supports this?

A Rely on the model remembering prior state after restart.

B Keep everything in the context window only.

C Have each agent export structured state to a known location.

✓ CORRECT

D Re-run the whole pipeline from zero with no state.



Answer C — Structured state exports plus a coordinator-loaded manifest on resume enable crash recovery without full re-exploration. The coordinator loads a manifest on resume and injects it.

You must migrate a library used across 45+ files with multiple viable integration approaches and differing infrastructure needs. Which mode is appropriate?

A Direct execution, changing files as you go — a step that introduces a new dependency for little gain.

B Direct execution with one big upfront instruction.

C Neither; require a human to do it manually.

D Plan mode, to explore and design before committing to changes.

✓ CORRECT



Answer D — Plan mode suits large-scale, multi-file, architecturally significant work (library migrations, multiple valid approaches), enabling safe exploration before changes.

SECTION



AI Fluency Framework

The 4Ds model, teaching approaches, and assessment methods

Q53-Q66

Which list correctly names the four competencies of the AI Fluency framework?

A Delegation, Description, Discernment, Diligence

✓ CORRECT

B Data, Design, Deploy, Document — a tempting shortcut that skips the underlying fix.

C Define, Decide, Deliver, Debug

D Discover, Describe, Decide, Defend



Answer A — The 4Ds are Delegation, Description, Discernment, and Diligence.

Deciding which parts of a task to give to AI versus do yourself, and how to divide the work, is which competency?

A

Description

B

Delegation

✓ CORRECT

C

Discernment

D

Diligence



Answer B — Delegation is about what to entrust to AI vs. yourself and how to share the work (problem, process, and platform awareness).

Communicating clearly with an AI — providing context, specifying the request, and shaping the interaction — is which competency?

A Delegation

B Discernment

C Description

✓ CORRECT

D Diligence



Answer C — Description is effective communication with AI: setting context, specifying the request, and structuring the exchange.

Critically evaluating AI outputs — judging quality and spotting errors — corresponds to which competency?

A Delegation

B Description

C Diligence

D Discernment

✓ CORRECT



Answer D — Discernment is the critical evaluation of AI products, processes, and behavior, rather than accepting output at face value.

Engaging with AI responsibly — being transparent about use and accountable for outcomes — is which competency?

A Diligence

✓ CORRECT

B Discernment

C Description

D Delegation



Answer A — Diligence is responsible, transparent, and accountable engagement: thoughtful creation, transparency, and deployment.

A user rewrites a vague prompt to add the audience, the desired format, and two examples of the output they want. Which competency are they practicing?

A

Delegation

B

Description

✓ CORRECT

C

Discernment

D

Diligence



Answer B — Adding context, format, and examples to a prompt to communicate the task clearly is Description.

Choosing not to use AI for a sensitive medical decision, and documenting that choice, most directly engages which pair?

A Description and Discernment

B Discernment and Description

C Delegation and Diligence

✓ CORRECT

D Delegation and Description



Answer C — Deciding whether (and what) to delegate is Delegation; transparent, accountable, responsible handling is Diligence.

An analyst prompts, evaluates the result, refines the prompt, and re-evaluates, iterating to a better answer. This refinement loop pairs which two competencies?

A Delegation and Diligence

B Discernment and Diligence

C Delegation and Description

D Description and Discernment

✓ CORRECT



Answer D — Crafting prompts (Description) and critically evaluating outputs (Discernment) together form the iterative refinement loop.

An instructor scores only the final AI-assisted report against a rubric, ignoring the collaboration process. What is this called?

A

Outcome-based assessment

✓ CORRECT

B

Process-based assessment

C

Reflection-based assessment

D

Peer assessment



Answer A — Outcome-based assessment judges the final product; it can miss how AI was used.

To evaluate collaboration skill, an instructor reviews a student's delegation choices, prompt iterations, and how they checked outputs. Which assessment type?

A Outcome-based assessment

B Process-based assessment

✓ CORRECT

C A timed quiz

D Automated grading only



Answer B — Process-based assessment examines how the learner works with AI — delegation, prompting iterations, and evaluation — not just the artifact.

Students write a short narrative explaining why they delegated certain tasks, how they verified results, and what they'd change. Which assessment type?

A Outcome-based assessment

B Process-based assessment

C Reflection-based assessment

✓ CORRECT

D Rubric-only scoring



Answer C — Reflection-based assessment asks learners to articulate their reasoning and self-evaluate their AI collaboration.

Why do educators favor realistic scenarios over definition-only quizzes when teaching AI fluency?

A Scenarios are the cheapest to grade — an approach that treats the symptom rather than the cause.

B They remove the need for human evaluation.

C They force every learner to one identical answer.

D They require applying the 4Ds together with judgment in context, beyond recalling terms.

✓ CORRECT



Answer D — Scenario-based learning makes learners apply Delegation, Description, Discernment, and Diligence together in realistic contexts.

An argument for weaving AI fluency into existing disciplinary courses rather than a single standalone module is that:

A Embedding it in authentic disciplinary tasks builds transferable, contextual judgment.

✓ CORRECT

B It eliminates the need to teach the discipline.

C It removes any need for assessment — a choice that looks simpler but shifts the problem elsewhere.

D It prevents students from ever using AI.



Answer A — Integration into authentic disciplinary work develops contextual, transferable judgment that a standalone module is less able to achieve.

A learner spots that an AI summary attributes a quote to a source that never said it, and corrects it. Which competency is shown?

A Delegation

B Discernment

✓ CORRECT

C Description

D Diligence

 Answer B — Detecting a fabricated or misattributed claim is the critical-evaluation work of Discernment.

W R A P - U P

Summary



Context & Escalation 18%

Stateless model, case-facts blocks, lost-in-the-middle, and when to escalate.



Batch API & Extraction 12%

50% cost vs 24h window, custom_id recovery, nullable schemas, retry limits.



Tool Design 21%

Descriptions drive selection; scope per role; typed errors; forced/any tool_choice.



MCP Design Patterns 18%

Project vs user scope, env-var secrets, resources vs tools, hooks for guarantees.



Agentic Systems 12%

Coordinator decomposition, parallel Task spawns, explicit context, crash recovery.



AI Fluency 19%

The 4Ds — Delegation, Description, Discernment, Diligence — and assessment types.

REMEMBER THIS

Key Takeaways



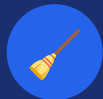
Hard rules need hard enforcement

When a question says "guarantee," "must," or "zero failures," choose programmatic hooks/prerequisite gates — not prompt wording.



Structure your handoffs & errors

Typed error metadata (errorCategory, isRetryable) and compact escalation summaries beat raw transcripts and generic failures.



Practice context hygiene

Trim verbose tool outputs, keep a persistent case-facts block, and counter lost-in-the-middle with summaries + section headers.



Escalate on the right triggers

Explicit human request, policy gap/silence, or no meaningful progress — not sentiment or self-reported confidence.



Tool selection lives in descriptions

Differentiate similar tools, scope each agent to its role, and prefer the proportionate first fix over new ML infrastructure.



Match the API to the latency

Batch API for overnight/latency-tolerant jobs; synchronous API for blocking, interactive, or multi-turn tool-calling work.

BEFORE YOU SIT

Exam Tips

- 1 Read the stem's exact ask** — Distinguish "most effective FIRST step" from "BEST/MOST effective" — the proportionate fix often wins over the maximal one.
- 2 Prefer determinism on "must"** — If a rule must hold every time (money, identity, ordering), pick hooks/gates over prompt instructions, which fail probabilistically.
- 3 Find the real root cause** — In multi-agent failures, blame the agent that made the decision (often the coordinator), not downstream agents working within their scope.
- 4 Reject over-engineering** — When prompt/description tweaks haven't been tried, a new classifier or ML pipeline is usually the wrong, heavier answer.
- 5 Classify errors precisely** — Separate transient vs validation vs business vs permission, and access failures vs valid empty results, before choosing recovery.
- 6 Let stop_reason drive loops** — Continue on "tool_use", stop on "end_turn" — never terminate on text phrases or arbitrary iteration caps.
- 7 Ignore answer length & letter** — Correct options here are randomized in position and length — decide on the concept, not the longest or most-qualified-sounding choice.